

1: Review the overview and create a bulleted list of requirements for this program

- 1) be able to correctly xor a plaintext with a ciphertext
- 2) be able to correctly decrypt an xor ciphertext given the correct key

2: Use your answer #1 to create a list of tests

- 1) given a simple ciphertext with 100 characters and a simple key, test if the program is able to xor correctly
- 2) given said ciphertext, test if the same key is able to decrypt the ciphertext successfully

3: Create a bulleted list of 5 requirements for this program that involve edge cases and/or error handling

- 1) In the event of receiving an extremely large plaintext, the code does not buffer overflow
- 2) In the event of the key not containing a null byte, the code does not read an infinitely large key
- 3) In the event of a key that is larger than 32 characters, the code should not read past the 32nd character
- 4) In the event of a filename that is larger than 100 characters, the code should not crash or buffer overflow
- 5) In the event of filename or key not ending in `\n`, `strcspn` does not cause the program to break

4: Use answer #3 to create a list of tests you need to perform to verify the edge cases

- 1) create and use a 1000 byte plaintext
- 2) create a key that does not end in a null byte
- 3) create a key that is 500 bytes long

5) encrypt a valid filename that does not end in \n or \0

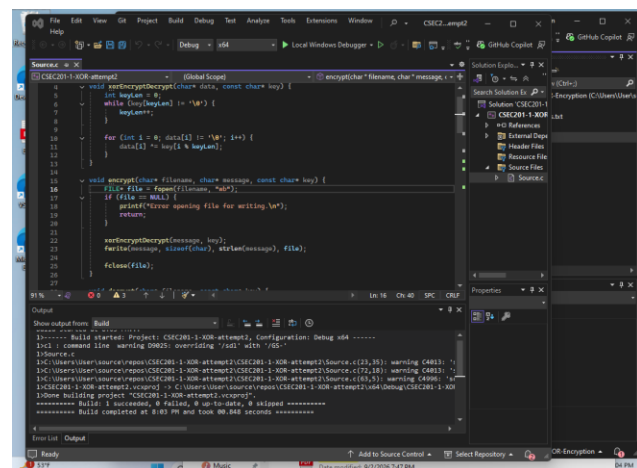
```
Solve.py 2 X EdgecaseTester.py 4 X StandardCiphertext.txt X edgecase1.txt X LargeFingerprint.png X CriticalThinkingQuestions.txt X Notes.txt X
CSEC201-Assignment-1-XOR-Encryption X EdgecaseTester.py X testEdgecase2

1 from pwn import *
2
3 XOR = process("./xor", stdin=process.PTY, stdout=process.PTY)
4
5 def testEdgecase1():
6     print(XOR.recvuntil(b": "))
7     XOR.sendline(b"e")
8     print(XOR.recvuntil(b": "))
9     XOR.sendline(b"edgecase1.txt")
10    print(XOR.recvuntil(b": "))
11    XOR.sendline(b"A"*1000)
12    print(XOR.recvuntil(b": "))
13    XOR.sendline(b"Z")
14
15 def testEdgecase2():
16    print(XOR.recvuntil(b": "))
17    XOR.sendline(b"d")
18    print(XOR.recvuntil(b": "))
19    XOR.sendline(b"StandardCiphertext.txt")
20    print(XOR.recvuntil(b": "))
21    XOR.send(b"abc123")
22    print(XOR.clean())
23    XOR.interactive()
24
25 def main():
26    # print(XOR.recvuntil(b": "))
27    # XOR.interactive()
28    # testEdgecase1()
29    testEdgecase2()

aidan@MSI: ~/CSEC201/CSEC X Ubuntu
aidan@MSI:~/CSEC201/CSEC201-Assignment-1-XOR-Encryption$ python3 EdgecaseTester.py
[+] Starting local process './xor': pid 5356
b'Do you want to encrypt or decrypt? (e/d): '
b'Enter the filename: '
b'Enter the decryption key: '
b''
[*] Switching to interactive mode
$
```

Test 1 failed due to being unable to accept any file input

Test 2 failed for the same reason



Both tests shared the same fix

7:

error on non edgecase test 1) cannot take file input and thus, unable to read from file or write to file.

solution: newline character from fgets was carrying over, so I had to add a quick getchar() in there to consume the newline

noticed that the encryption program will also encrypt the newline character at the end of a ciphertext. This isn't necessarily an error, but it's good to note

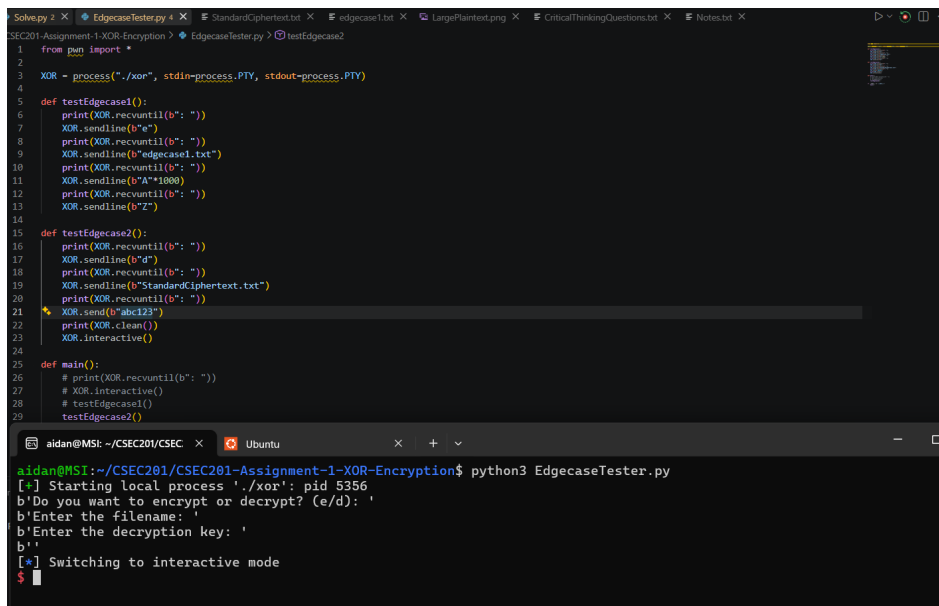
all other standard test cases past correctly, moving onto edge case test cases

8:

edgecase test 1 does not write to file if input is greater than 256 bytes. (see edgecase1.txt in the repository)

edgecase test 2 does not output a plaintext if key does not contain a newline or null byte

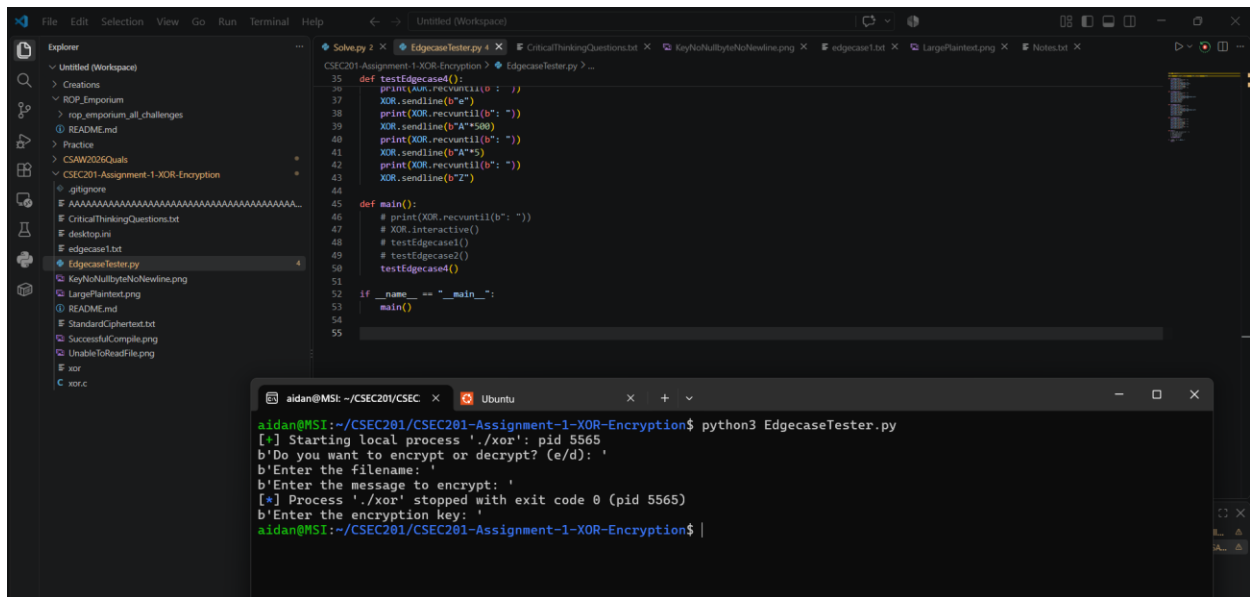
edgecase test 3 does not attempt to decrypt plaintext if key is greater than 32 bytes



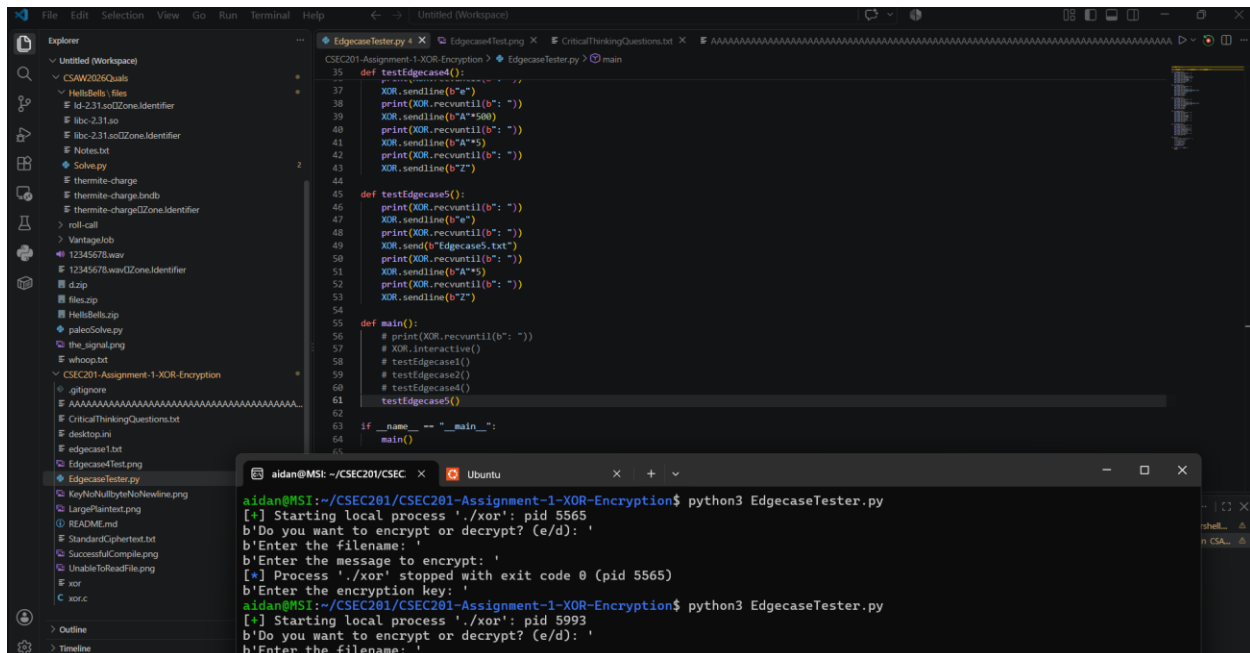
```
1 from pwn import *
2
3 XOR = process("./xor", stdin=process.PTY, stdout=process.PTY)
4
5 def testEdgecase1():
6     print(XOR.recvuntil(b": "))
7     XOR.sendline(b"e")
8     print(XOR.recvuntil(b": "))
9     XOR.sendline(b"edgecase1.txt")
10    print(XOR.recvuntil(b": "))
11    XOR.sendline(b"A"*1000)
12    print(XOR.recvuntil(b": "))
13    XOR.sendline(b"Z")
14
15 def testEdgecase2():
16    print(XOR.recvuntil(b": "))
17    XOR.sendline(b"d")
18    print(XOR.recvuntil(b": "))
19    XOR.sendline(b"StandardCiphertext.txt")
20    print(XOR.recvuntil(b": "))
21    XOR.send(b"abc123")
22    print(XOR.recvuntil(b": "))
23    XOR.sendline(b"")
24
25 def main():
26    # print(XOR.recvuntil(b": "))
27    # XOR.interactive()
28    # testEdgecase1()
29    testEdgecase2()

aidan@MSI: ~/CSEC201/CSEC x Ubuntu
aidan@MSI:~/CSEC201/CSEC-Assignment-1-XOR-Encryption$ python3 EdgecaseTester.py
[*] Starting local process './xor': pid 5356
b'Do you want to encrypt or decrypt? (e/d): '
b'Enter the filename: '
b'Enter the decryption key: '
b''
[*] Switching to interactive mode
$
```

edgecase test 4 creates a file with a maoe of 100 As, however no encrypted text is written to the file



edgecase test 5 fails to create a file and freezes if the file does not end with a null byte or newline



9:

chatGPT solved several edge cases (mostly regarding reading inputs that were too large) by creating a new readline function

this function reads the input and, should the input exceed the size, doubles the size through realloc to ensure any sized input can be function

it also uses a while loop that terminates at EOF to read the inputs in order to handle inputs that do not end in a null byte or a newline character

Additionally, chatGPT rejects keys of length 0, and also now preserves null bytes that occur in the ciphertext by reading based on file size

This was not one of my test cases but it probably should have been

10:

realloc is new, but it makes sense to put variables which you want to be arbitrarily large into the buffer so you can reallocate memory as needed for their size

Lots of other things that look like global constants such as EXIT_FAILURE are also new but self explanatory

ftell is also new. It essentially tells you how far you are from the beginning of a file. The way it is being used is after fseek the EOF so its basicly just finding the file size dynamicly

11:

A lot of the functionality around input was changed due to the dynamic reallocation and there were more minor adjustments made to a lot of the other components

however, I don't think the a whole lot of the core application behavior changed asside from the dynamic allocation of inputs

to me, the dynamic allocation seems rather unessecary, absolutely overkill for an issue like this, and possibly presents some if its own issues too

If it were up to me, I would have added a simple check to see if inputs were too big and exit the code if it is, not create an entirely new system.

The newly introduced problem that I foresee is putting in an absurdly large input that would break malloc or force too much allocated memory.

Finally, I'm unsure why chatGPT decided to use messageLen on everything rather than strlen(data).

Apparently chatGPT thinks thats important to do

12: I got grace's repo to compare mine to.

At the initial step, mine differs from hers in that XOR encryption and decryption was bundled into one function for mine but remains in two functions for her

my version also has a lot more strcspn blocks to turn newline characters into null bytes

additionally, my version prints the decrypted text while hers outputs it into a new file

at step 5, our fixes differed too as grace's encryption and decryption functions were broken

while mine only had problems accepting inputs

at the last step, my code has a weirder way of reading arbitrarily large input, while grace's code

just throws an error if the input is too large.

Additionally, hers creates an encrypted copy of the plaintext file should the plaintext be needed later